Atividades Estrutura de dados

🔷 1. FILAS (Queue – FIFO)

Exercício 1 — Simulador de Atendimento

Objetivo: Implementar fila usando array.

Enunciado

Implemente uma fila com operações:

- `enqueue(valor)`
- `dequeue()`
- `peek()`
- `isEmpty()`

Simule um sistema de atendimento bancário.

Complexidade Esperada

Operação	Complexidade
enqueue	O(1)
dequeue	O(1)
peek	O(1)

Pergunta de análise

Se a fila fosse implementada deslocando os elementos no `dequeue`, qual seria a complexidade?

Exercício 2 — Fila com Prioridade Simples

Implemente uma fila onde:

- Idosos entram antes
- Demais entram ao final

Discussão

- Você está mantendo ordenação?
- Está inserindo no meio?

Complexidade Esperada

Operação	Complexidade
enqueue	O(n)
dequeue	O(1)

Exercício 3 — Fila com Prioridade Simples inserção constante

Implemente uma fila onde:

- Idosos entram antes
- Demais entram ao final

Discussão

- Você está mantendo ordenação?
- Está inserindo no meio?

Complexidade Esperada

Operação	Complexidade
enqueue	O(1)
dequeue	O(1)

🔷 2. PILHAS (Stack – LIFO)

Exercício 3 — Validador de Parênteses

Enunciado

Verifique se uma expressão possui parênteses balanceados:

```
((a+b)*c)
```

Discussão Guiada

- Quando empilha?
- Quando desempilha?
- Como detecta erro?

Complexidade Esperada

Operação	Complexidade
Percorrer string	O(n)
push/pop	O(1)

Complexidade total: **O(n)**

Exercício 4 — Inversão de Texto com Pilha

Inverter uma string usando pilha.

Complexidade Esperada

Etapa	Complexidade
Empilhar	O(n)
Desempilhar	O(n)
Total	O(n)

🔷 3. LISTAS ENCADEADAS

Exercício 5 — Implementar Lista Encadeada Simples

Criar estrutura Node:

```
valor
next
```

Operações:

- inserir no início
- inserir no final
- remover por valor

Complexidade Esperada

Operação	Complexidade
Inserir início	O(1)
Inserir final (sem tail)	O(n)
Remover	O(n)
Buscar	O(n)

Exercício 6 — Lista com Ponteiro Tail

Adicione ponteiro para último elemento.

Complexidade Esperada

Operação	Complexidade
Inserir final	O(1)

Exercício 7 — Detectar Ciclo (Introdução)

Implementar algoritmo de Floyd (tartaruga e lebre).

Complexidade Esperada

Tempo: **O(n)**

Espaço: **O(1)**

🔷 4. BUSCA BINÁRIA

Exercício 8 — Implementar Busca Binária Iterativa

Array ordenado.

Enunciado

Buscar número dentro de array ordenado.

Complexidade Esperada

Caso	Complexidade
Melhor caso	O(1)
Médio/Pior	O(log n)

Exercício 9 — Busca Binária Recursiva

Comparar versão iterativa vs recursiva.

Complexidade Esperada

Tempo: **O(log n)**

Espaço (pilha recursiva): **O(log n)**

Exercício 10 — Comparação Prática: Busca Linear vs Binária

Dado:

- Array com 1.000.000 elementos ordenados

Simular número máximo de comparações.

Complexidade Esperada

Algoritmo	Complexidade
Linear	O(n)
Binária	O(log n)

Discussão Matemática:

$\log_2(1.000.000) \approx 20$

🔷 Exercícios Integradores

Exercício 11 — Simulador de Navegador (Pilha + Pilha)

Implementar:

- Voltar
- Avançar

Complexidade:

Todas operações **O(1)**

Exercício 12 — Sistema de Impressão (Fila + Prioridade)

Simular impressora:

- Documentos entram na fila
- Documento urgente passa na frente

Complexidade esperada:

Inserção: O(n)

Remoção: O(1)

🔷 Exercício Avançado

Exercício 13 — Comparação de Estruturas

Dado o cenário:

> Sistema precisa:
>
> - Inserir frequentemente
> - Buscar frequentemente
> - Remover ocasionalmente

Qual estrutura escolher?

Discussão:

- Lista?
- Array?
- Fila?
- Pilha?

Complexidade comparativa:

Estrutura	Inserção	Busca	Remoção
Array	O(n)	O(1)	O(n)
Lista Encadeada	O(1)	O(n)	O(n)
Busca Binária (array ordenado)	O(n)	O(log n)	O(n)